

ELTE EÖTVÖS LORÁND UNIVERSITY

DATA MANAGEMENT WITH CRUD

Modern web application development in .NET

Lecture 7

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

CRUD

In many cases the web service provides basic data processing operations, the *CRUD* operations: *Create, Read, Update, Delete*

- these operations have HTTP equivalents (create: **`POST`**, read: **`GET`**, modify: **`PUT`**, delete: **`DELETE`**)
  - upon a successful operation the response code is **`CREATED`** (201) for creation, **`OK`** (200) or **`NO CONTENT`** (204) for other operations
    - with the **`CREATED`** result the created resource (object) and its location (**`GET`** path) must be returned as well
  - when the operation is not executed immediately, the status **`ACCEPTED`** (202) may be indicated
- in a RESTful service, the operations must conform to this scheme

DATA SYNCHRONIZATION MODELS

Client-side data management can be implemented in two ways:

- *synchronous:* client and server state are always the same
- *asynchronous:* client and server state are different and can be synchronized manually (save, load, update)

Asynchronous data management is advantageous if you want to save your changes in groups rather than individually

- to achieve this, changes have to be tracked on the client side with *status flags* and indicate what changes have been made to the data (new, modified, deleted)

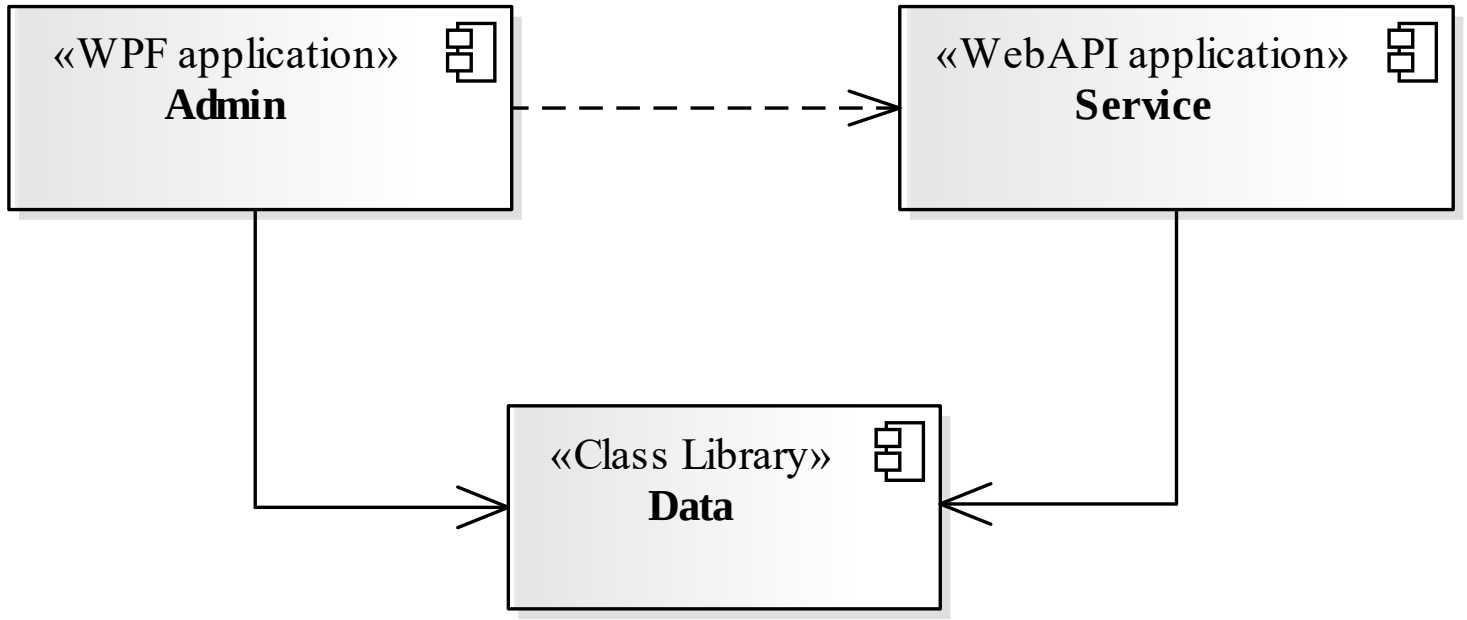
EXAMPLE

Task: Implement a desktop admin application for the travel agency website with the ASP.NET Core based webservice and a *WPF frontend*.

- The client will be a WPF application (**`TravelAgency.Admin`**) that will connect to an ASP.NET Core webservice (**`TravelAgency.WebAPI`**)
- the client application will be built in *MVVM architecture*, where persistence (**`TravelAgencyServicePersistence`**) will provide the network communication
    - the persistence layer is placed in a separate class library project (**`TravelAgency.Admin.Client`**) for easier testing and reusability
- Separate DTO types used for data transfer are placed in a separate class library (**`TravelAgency.Shared`**), and shared between the server and the client project to avoid code redundancy

EXAMPLE

Design (*architecture*):

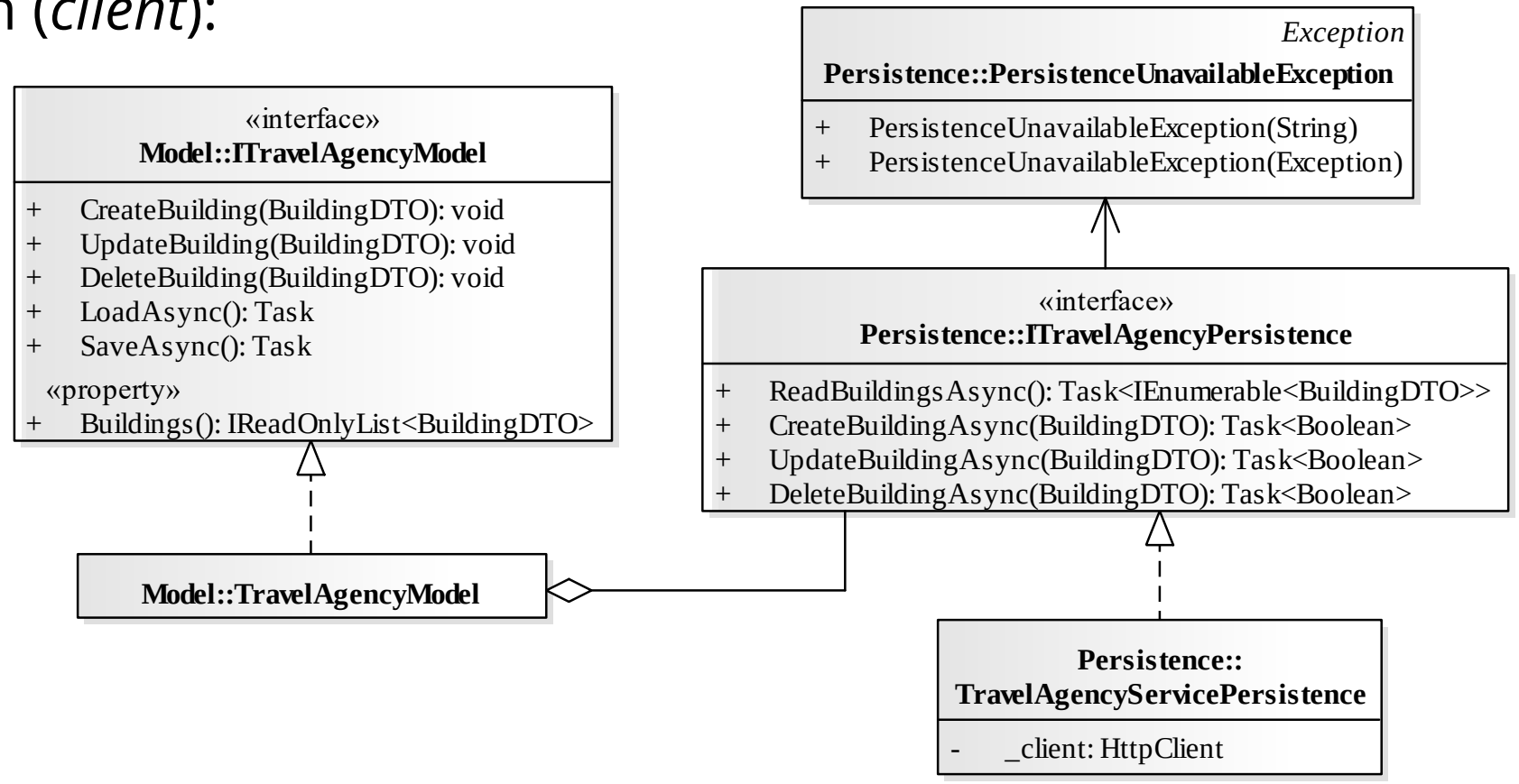


EXAMPLE

- The admin client should be able to add, edit and remove buildings
  - the client provides asynchronous data management, the model (**`ITravelAgencyModel`**) monitors the client state with status flags (**`DataFlag`**), based on which the appropriate action (add / update / delete) can be performed upon saving
  - persistence (**`ITravelAgencyPersistence`**) is responsible for loading, saving data with asynchronous operations
  - a temporary identifier is created for new buildings, which is replaced by a permanent identifier returned by the server when modifications are persisted to the web service
  - the **`BuildingsController`** of the service (**`TravelAgency.WebAPI`**) is extended with the missing CRUD operations

EXAMPLE

Design (*client*):

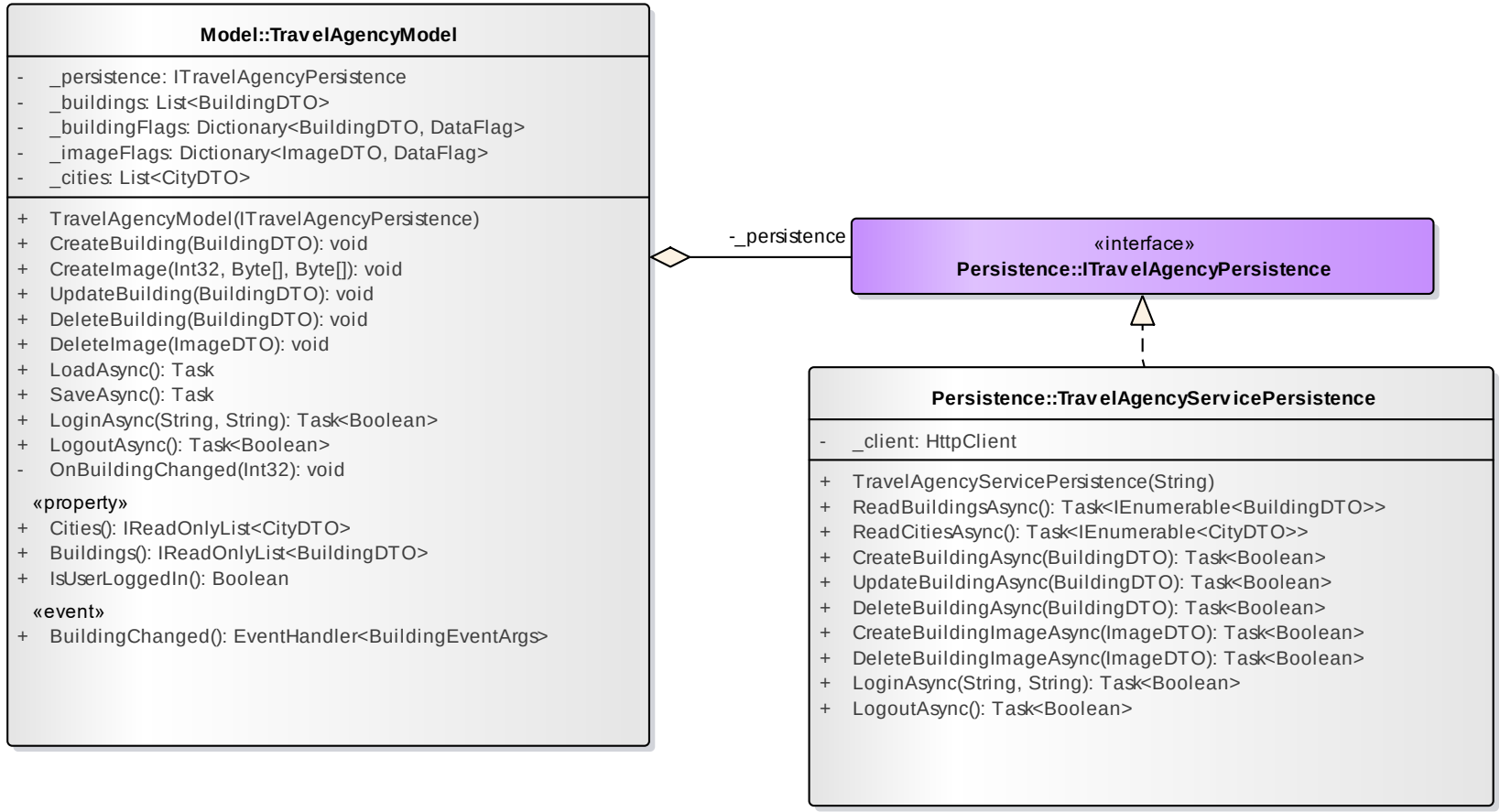


EXAMPLE

- The admin client should also support to view, add and delete pictures for buildings
  - images will be loaded from files, then resized and converted accordingly before upload (small and large size, PNG format)
  - images will be linked to buildings and also have a temporary unique identifier, until their permanent identifier is defined and returned by the webservice
  - the **`BuildingsController`** of the webservice is extended
- To ensure security, data management is protected by authentication and authorization (using *ASP.NET Core Identity*), so that the user must first log in to the application
  - the existing **`UsersController`** is used for login and logout

EXAMPLE

Design (*client*):



EXAMPLE

- Extend testing with the CRUD operations in all 3 test classes (**`BuildingsControllerTest`**, **`BuildingsMockServiceTest`**, **`BuildingsIntegrationTest`**)
  - In integration testing test the persistence layer of the client, e.g.:

```
var client = server.CreateClient();
// instantiate client
var persistence =
  new TravelAgencyServicePersistence(_client);
// instantiate client-side persistence layer
IEnumerable<BuildingDto> result =
  await persistence.ReadBuildingsAsync();
```

  - During integration testing, test the successful and unsuccessful authorization as well!